



PRENÒTATI

PROGETTO PROTOTIPO

PROPONENTI

Lo studente modello	0108/1234
La studentessa modello	0124/1234

DATA DI CONSEGNA

01/09/2014

ANNO ACCADEMICO

2013 - 2014

CATEGORIA

Ufficio

Indice

Metadati	v
Informazioni generali	v
Requisiti di complessità minima	viii
Note conclusive	ix
Bibliografia	x
Progettazione	2
Sintesi dei requisiti	2
Glossario	3
Diagramma EE/R	3
Diagramma relazionale	3
Utenti e loro categorie	4
Operazioni degli utenti	4
Volumi	6
Vincoli di integrità	7
Statici	7
Dinamici	7
Verifica di normalità	7
Possibili estensioni	8
Implementazione	11
Creazione utenti	11
Data Definition Language	11
Ricevendo	12
Ricevimento	12
Prenotazione	13
Data Manipulation Language	14
Trigger	16
Hashing	16
Classificazione	16
Procedure e funzioni	17
Programmazione ricevimento	17
Prenotazione	18
Disdetta	19
Conteggio dei posti disponibili	20
Viste	21
Ricevimenti disponibili	21
Prenotati	21
Data Control Language	22
Scheduler - facoltativo	22

Elenco delle figure

1	Diagramma Entità-Associazione	3
2	Diagramma relazionale	4
3	Diagramma UML dei casi d'uso	5

Elenco delle tabelle

1	Tavola degli utenti.	4
2	Tavola delle operazioni. B sta per <i>batch</i> e I per <i>interactive</i> . Usando solo SQL statico, tutte le operazioni sono <i>batch</i>	6
3	Tavola dei volumi. E sta per <i>entità</i> e ED per <i>entità debole</i> . Per le tabelle di transizione si indica una A, che sta per <i>associazione</i>	6
4	Scala ordinale dei livelli di difficoltà.	17

Elenco degli script

CreazioneUtenti.sql	11
RicevendoDDL.sql	12
RicevimentoDDL.sql	13
PrenotazioneDDL.sql	13
RicevendoDML.sql	14
RicevimentoDML.sql	15
PrenotazioneDML.sql	15
HashingTrigger.sql	16
ClassificazioneTrigger.sql	16
ProgrammazioneRicevimentoProc.sql	17
PrenotaProc.sql	18
DisdiciProc.sql	20
ContaSlotFunc.sql	20

RicDispView.sql	21
PrenotatiView.sql	21
DiscenteDCL.sql	22
DocenteDCL.sql	22
RolloutScheduler.sql	22

Metadati

HO DECISO DI pubblicare un progetto prototipo che deroga ai requisiti di complessità minima¹ e tutto il codice sorgente per la creazione, il popolamento e il *testing* del relativo DB per fornire un esempio conciso, chiaro e fruibile di come deve essere organizzato un buon progetto: auspico che il mio sforzo germini progetti di qualità elevata e con tempi di gestazione più brevi.

Un caso reale apparentemente semplicissimo — la gestione del ricevimento di un singolo docente — ha prodotto alcuni problemi di complessità sorprendente, senza soluzioni ovvie e che richiedono strumenti non presentati nel primo corso (come gli indici, il FOR UPDATE, l'hashing o le espressioni regolari) per essere risolti. Si tratta di problemi comuni a molte applicazioni, che riguardano la gestione della concorrenza, il controllo degli accessi, i vincoli di tupla, i tipi nativi e la sicurezza, e la cui comparsa corrobora il mio vecchio motto che nulla è banale. Ciò mi ha posto di fronte ad una scelta: nascondere la complessità come la polvere sotto un tappeto, presentando una versione *ad usum delphini* del DB, oppure presentare soluzioni efficaci, ma non immediatamente comprensibili, in una versione credibile del DB stesso. Ho scelto di presentare la realtà grossomodo per come è: complessa, spigolosa e ostile. Sebbene non pretenda altrettanto (e segnatamente l'uso di strumenti non trattati durante il corso), spero che pur con tutti i suoi *handicap* questo progetto sia uno spunto utile a dilatare il punto di vista dei futuri sviluppatori di applicazioni.

Nel primo capitolo sono raccolte informazioni generali utili alla stesura: istruzioni e suggerimenti ad ampio spettro che vanno dalla scelta del tema alle modalità di consegna.² Il secondo capitolo espone la progettazione del Database PrenòTaTi, partendo dalla raccolta dei requisiti e giungendo alla sua normalizzazione. Il terzo capitolo tratta l'implementazione, ovvero la trasformazione in codice di quanto descritto nel capitolo precedente, e contiene frammenti eseguibili opportunamente commentati. Il quarto capitolo infine è dedicato al collaudo, ed è in via di definizione.

È possibile saggiare la complessità di qualsiasi Database [1, 2] osservando che il DB qui proposto è nato per sostituire un elenco cartaceo di quindici persone.

n.b. Il documento qui presente è coperto da *copyright* ed è utilizzabile solo dagli studenti dell'Università Parthenope di Napoli come supporto alla preparazione dell'esame di Basi di Dati. Non ne sono consentite né la duplicazione, né l'alterazione, né la pubblicazione, parziale o totale.

Informazioni generali

Il tema è libero e suggerisco di cimentarsi con un caso reale, adottando il negozio dello zio, la palestra del cugino o la biblioteca del nonno. Anziché sforzare la fantasia nell'immaginare

¹Non sono rispettati i numeri minimi di entità, trigger e procedure; nella sua forma attuale questo progetto non sarebbe sufficiente per l'ammissione, pur essendo ben fatto.

²Il contenuto di questo capitolo è proprio del progetto prototipo.

come potrebbe funzionare una realtà che non si conosce, un caso reale fornisce il preziosissimo supporto di un termine di confronto e di un'interfaccia umana alla quale porre le domande. È il modello che si deve adattare alla realtà e non viceversa — si rifugga da questa tentazione. Temi inflazionati sono il negozio di informatica, il campionato di calcio, il concorso musicale: si evitino. L'originalità è oggetto di valutazione e incide positivamente su quest'ultima.

Contesto

Attenzione al contesto: se si progetta un DB per la vendita di oggetti e si usufruisce di un'unica società di spedizione per le consegne, è rilevante memorizzare solo le informazioni sulle consegne dei pacchi (quale pacco è stato consegnato, al e dal corriere, e a che ora); chi siano i dipendenti della società di spedizione, quali i mezzi usati e i tragitti seguiti è un'informazione utile alla società di spedizione stessa, irrilevante per il DB della società di vendita. Se quest'ultima usufruisce di più corrieri sarà rilevante per il progettista tracciare qual è la società di spedizione a cui si affida ciascun pacco (è sufficiente un identificativo univoco) e se e quando sia stato consegnato, a maggior ragione il DB della società di vendita non includerà le informazioni su tutti i dipendenti, mezzi e tragitti di tutti i corrieri di cui si avvale.³ Se si modella il DB di una società di spedizione vale il contrario.

Se il database è di una singola biblioteca, “biblioteca” non è un'entità; se è di una singola squadra di calcio, allora quella squadra non comparirà nel database, né come entità, né come tupla, né come attributo; se è di un singolo albergo, allora l'indirizzo dell'albergo e tutti i relativi dati identificativi non compariranno nel DB; ancora, se per l'attività modellata esiste un solo magazzino o un solo fornitore, allora “magazzino” o “fornitore” non sono entità. Si guardino con sospetto le entità con un solo attributo, poiché è verosimile che possano essere trasformate in attributi semplici senza accettare anomalie o con un minimo compromesso. Tutto ciò che è univoco e ovvio dato il contesto deve essere eliminato dall'E/R.⁴

Esiste una differenza concettuale e sostanziale tra gli utenti del DBMS e i clienti, i dipendenti o le persone fisiche i cui dati a qualunque titolo sono presenti nel DB. Di norma gli utenti della base di dati non costituiscono un'entità dell'E/R, sono gestiti attraverso tabelle di sistema, e non esiste alcuna corrispondenza tra loro e le tuple di una qualsiasi relazione. Si può memorizzare in una relazione un elenco di clienti che non avranno mai accesso alla Base di Dati, senza quindi creare alcun utente corrispondente, oppure si può creare un utente unico con ruolo “cliente” che permette l'accesso a tutti, oppure si potrebbe creare molti utenti di tipo “cliente” con permessi diversi e abilitarli con regole e tempi diversi, indipendentemente dalle tuple memorizzate nella relazione “cliente”. La gestione degli utenti è fatta dall'amministratore del DB, e questi sono memorizzati in un'apposita tabella di sistema.⁵

Redazione

Ottimi manuali con indicazioni redazionali chiare e utili sono il testo di Roberto Lesina [3] o il *New Oxford style manual* [4] per gli esteroфи. Incoraggio l'uso di L^AT_EX (per chi vuole un buon punto di partenza è [5]), ma ammetto anche l'indegno *Ms Word*, purché il risultato sia conforme, per quanto possibile, a questo documento.

Il frontespizio deve riportare il titolo del progetto, le informazioni essenziali per identificare i proponenti, la categoria del tema trattato (ad esempio trasporti, comunicazioni, telefonia, bibliofilia, ufficio, vendite all'ingrosso o al dettaglio etc.), l'anno accademico, la data di consegna,

³Chiamiamo l'inclusione erronea di un DB in un altro *over-scope*.

⁴Chiamiamo l'introduzione di entità e attributi ovvi nell'E/R *self-scope*.

⁵Chiamiamo l'identificazione di clienti e utenti *cross-scope*.

un'immagine. È obbligatoria la presenza di un indice dettagliato dei contenuti ed è opportuno cercare di limitare il numero di pagine usando interlinea singola, caratteri a spaziatura proporzionale e margini entro i tre centimetri.

Esiste stile anche nella stesura del codice e nella programmazione, e i manuali di *Software Engineering* possono essere visti come guide in tal senso; segnalo il buon Pressman [6], che resiste bene agli acciacchi dell'età. Scrivere codice efficace, chiaro, modulare, leggibile e riutilizzabile è un'arte: siate bravi artigiani. Nel caso specifico di PL/SQL, raccomando di usare poco i cursori e di produrre codice dichiarativo, non procedurale. I motori DBMS sono ottimizzati per SQL e usare i cursori significa costringerli in una logica procedurale che ne rallenta enormemente le prestazioni, oltre a dimostrare di aver mancato il senso di questo corso (è diagnosticabile una *C-addiction*).

Commentate il codice! Preferite un *editor* che evidenzi la sintassi e usate un'indentazione sensata, atta a facilitare la lettura e la comprensione da parte di chi legge.

Sebbene sembri superfluo, anni di esperienza mi hanno insegnato che è quanto mai necessaria una pulizia nello stile della scrittura e una bella ripassata di italiano. Errori e orrori ortografici punteggiano i progetti di BD in varia misura e, sebbene non comportino penalizzazione, sarebbe opportuno diradarli. Non posso far altro che raccomandare il salubre manuale di Massimo Birattari [7].

Stampa

Il progetto va stampato su carta liscia formato A4, una sola facciata per foglio, e rilegato con una spirale. Se l'E/R o il relazionale risultassero poco leggibili per via delle dimensioni, vanno stampati in formato A3 e ripiegati in due. Le pagine con il codice sono preferite a colori, sempre che si sia usato un *editor* con evidenziazione della sintassi.

Sul CD/DVD vanno scritti con un pennarello indelebile Il titolo del progetto, nome, cognome e matricola dei proponenti, l'anno accademico, la data di consegna e la categoria generale. Il CD va organizzato in tre cartelle: *Documentazione*, contenente la relazione in formato pdf e sorgente (TeX, odt, doc, etc.); *Codice*, eventualmente suddivisa in sottocartelle, con tutto il codice sottoforma di script; *Dump*, facoltativa, con il *dump* del DB e un file *leggi.txt* con le credenziali per il ripristino. È opportuno separare gli script di creazione e popolamento.

Consegna

La consegna del progetto, ovvero della relazione cartacea con CD/DVD in allegato, deve avvenire entro la data indicata di volta in volta tramite avviso sulla piattaforma di e-learning, ed in ogni caso prima della prova pratica. Solo coloro che hanno già superato la prova pratica e devono sostenere l'esame orale possono consegnare il progetto il giorno stesso della prova pratica. La consegna può essere anticipata o avvenire via posta ordinaria, purché il plico arrivi nei tempi indicati (va da sé che la consegna con questa modalità avviene a rischio del mittente). In nessun caso sarà ammessa la consegna via email.

Il progetto resta valido per tre appelli utili consecutivi, dopodiché può essere prorogato al costo di modifiche ed integrazioni. Gli appelli a cui un candidato non può partecipare non vanno conteggiati.⁶

È possibile consegnare in orario di ricevimento al docente, oppure al sig. Franco Rossi nella stanza 416 in orario d'ufficio, o in ultima analisi alla portineria.

⁶Per ulteriori informazioni sulla validità si consulti la sezione F.A.Q. sul sito del corso.

Correzione

La valutazione massima è di 7 punti e la minima per l'ammissione è di 3; è comune anche una valutazione oscillante tra due punti, ad esempio 4/5. La valutazione del progetto è unica e il punteggio ottenuto vale per tutti i proponenti.

I criteri usati per valutare un progetto sono:

- adeguatezza al contesto del modello concettuale e della sua traduzione in relazionale;
- originalità;
- complessità (sensata, non onanismo);
- qualità della documentazione allegata.

Per raggiungere il punteggio pieno, oltre a superare i requisiti minimi, usualmente — ma non necessariamente — è richiesto l'utilizzo dello *Scheduler*⁷, argomento non trattato durante il corso.

La correzione avverrà solo dopo il superamento della prova pratica e solo in seguito a questa saranno resi noti i risultati.

Requisiti di complessità minima

Scelto un tema, il primo passo è la raccolta dei requisiti in linguaggio naturale. Ciò è effettivamente realizzabile tramite interviste e analisi del sistema preesistente se si è scelto un caso reale, altrimenti va simulato producendo un testo che somigli alla traccia di un esercizio di modellazione concettuale.

Il diagramma EE/R deve comprendere minimo K entità, contando una sola volta le entità che producono specializzazioni (ovvero escludendo dal conteggio le entità figlie dovute alle specializzazioni). E' facoltativo partire con gli schemi scheletro di settore fusi poi in uno schema unico. I requisiti raccolti devono includere anche aspetti strutturali, dinamici e quantitativi riguardanti le categorie di utenti, i privilegi di accesso di ciascuna, le prestazioni attese, il numero di potenziali utenti della base di dati e la dimensione stimata delle tabelle a regime.⁸ Inoltre vanno indicati i vincoli di integrità statici e minimo P vincoli di integrità dinamici, le operazioni di base (quelle che coinvolgono una sola tabella) e minimo Q operazioni degli utenti (quelle che coinvolgono più tabelle: è fortemente preferibile che comportino operazioni di scrittura e non siano semplici interrogazioni), ciascuna che coinvolga minimo R tabelle. Si fa presente che i vincoli di integrità dinamici e le operazioni degli utenti possono essere anche ottenuti da regole di business e quindi di fatto imposti dal progettista (ad esempio si può stabilire che un supervisore non possa avere più di 10 sottoposti o che non si effettuino vendite nel periodo di ferragosto). Le Q operazioni degli utenti vanno ulteriormente specificate attraverso i diagrammi use case e compilando la scheda del descrittore di operazione (si veda l'Albano [2] a pag. 51). Lo schema concettuale va quindi trasformato in uno schema relazionale (ovvero lo schema contenente soltanto le tabelle e le frecce che collegano le chiavi esterne alle chiavi primarie) secondo le regole viste durante il corso e va creato un glossario che definisca i termini principali e/o in qualsiasi modo ambigui. Va effettuata infine la verifica della normalità dello schema proposto onde appurare se sia in BCNF o almeno in 3NF e nel caso sia presente qualche dipendenza anomala segnalarla o risolverla.

⁷ Trigger che si attiva sulla base di eventi temporali.

⁸ Questi dettagli sono essenziali per la progettazione fisica, non trattata nel primo corso.

Numero studenti	K	P	Q	R
1	7	4	3	3
2	10	6	4	3
3	12	8	5	3

Tutte le tabelle e i vincoli individuati nello schema relazionale vanno implementati in Oracle 10g XE o 11g XE con le opportune istruzioni SQL DDL. Le tabelle vanno quindi popolate con dati fittizi (minimo 15 record per tabella e in ogni caso tanti da permettere la verifica del corretto funzionamento delle procedure implementate). Va creato almeno un utente per ogni categoria d'utenti e bisogna garantire tramite le opportune viste che egli acceda solo ai dati previsti in fase di progettazione. I P vincoli di integrità dinamici vanno implementati tramite P triggers in PL/SQL, mentre le Q operazioni degli utenti vanno implementate attraverso Q *stored procedures* (o funzioni, se del caso) sempre in PL/SQL. Sia nei *triggers* che nelle *stored procedures* vanno gestite esplicitamente le eccezioni, mentre le seconde vanno programmate come transazioni esplicite (ovvero occorre gestire i comandi di COMMIT e ROLLBACK).

Elenco documentazione:

- sintesi dei requisiti;
- glossario;
- diagramma EE/R;
- elenco delle operazioni di base;
- elenco delle operazioni degli utenti;
- Use case e descrittore di operazione per le operazioni degli utenti;
- elenco dei vincoli d'integrità statici;
- elenco dei vincoli di integrità dinamici;
- diagramma relazionale;
- verifica di normalità dello schema;

Note conclusive

I progetti non conformi a queste regole riceveranno uno o più punti di penalizzazione, a mia discrezione e come monito per lavoro superficiale o malfatto.

Non mi assumo alcuna responsabilità, né suggerisco o incoraggio in alcun modo il rilascio, ovvero l'uso dei progetti come software idoneo ad un'applicazione reale. Le finalità sono meramente didattiche e giocoforza restano non modellati o testati adeguatamente troppi aspetti per confidare che il codice prodotto si possa effettivamente usare in un sistema reale.

Raccomando di non ispirarsi troppo fedelmente al lavoro di altri, poiché il plagio è una forma di truffa e lo tratterò come tale. Considero preferibile un progetto non eccelso ma originale ad un progetto inappuntabile non frutto del proprio lavoro. Scommettete su voi stessi e siate fiduciosi.

Buon lavoro
Antonio Maratea

Bibliografia

- [1] Ramez A. Elmasri, Shamkant B. Navathe, “Sistemi di basi di dati - fondamentali”, Pearson, 2007.
- [2] Giorgio Ghelli, Renzo Orsini, Antonio Albano, “Fondamenti di basi di dati”, Zanichelli, 2005.
- [3] Roberto Lesina, “Il nuovo manuale di stile. Guida alla redazione di documenti, relazioni, articoli, manuali, tesi di laurea”, Zanichelli, 2009.
- [4] AA.VV., “New Oxford style manual”, Oxford University Press, 2012.
- [5] Lorenzo Pantieri e Tommaso Gordini, “L’arte di scrivere con L^AT_EX”, 2012.
http://www.lorenzopantieri.net/LaTeX_files/ArteLaTeX.pdf
- [6] Roger Pressman, “Principi di ingegneria del software”, McGraw-Hill, 2008.
- [7] Massimo Birattari, “È più facile scrivere bene che scrivere male. Corso di sopravvivenza”, Ponte alle Grazie, 2011.

Progettazione

Requisiti
EE/R
Relazionale
Utenti
Operazioni
Volumi
Vincoli
Normalizzazione
Estensioni

In questo capitolo si tratta la progettazione del DB. I requisiti sono raccolti, dissezionati e documentati in linguaggio naturale, con l'ausilio di vari diagrammi e tabelle.

Progettazione

"Calcolate la stima migliore del tempo che serve per realizzare un progetto, se tutto va bene dovrete moltiplicarla per due, altrimenti per quattro."

MONSIEUR DE LA PALISSE, temibile docente di Ovvietà I, II e III, vuole automatizzare la gestione del suo orario di ricevimento. Da un'intervista diretta e dalla rielaborazione di quanto detto, sono scaturiti i requisiti seguenti.

Sintesi dei requisiti

Il database deve gestire le prenotazioni per il ricevimento studenti di un singolo docente. Il ricevimento si tiene di norma ogni martedì pomeriggio, dalle 14:30 alle 17:30, ma talvolta si tiene, previo avviso e per i motivi più vari, in altri giorni o orari. Gli studenti devono potersi connettere al database, prenotare ed eventualmente disdire la prenotazione in totale autonomia, senza intervento del docente stesso. Il docente deve poter fissare data, ora e durata dei vari ricevimenti e consultare l'elenco dei prenotati per il prossimo ricevimento. Prima che uno studente possa prenotarsi per un qualsiasi ricevimento, deve mandare una email al docente in cui fornisce i suoi dati personali e chiede di essere inserito nell'elenco dei ricevendi. In seguito alla ricezione dell'email il docente carica i dati dello studente nel DB e manda una email di conferma allo studente comunicandogli il suo *hid*, ovvero il codice personale che lo studente dovrà usare per prenotarsi. Il ricevimento è suddiviso in *slot* di durata fissa di 10 minuti, ed un ricevendo può prenotarne uno solo per giorno. In un giorno ci sono al massimo 20 incontri e minimo 3. Il docente riceve tanto studenti, quanto tesisti, dottorandi o altri. Esiste una classificazione dei tipi di ricevimento ("standard", "hard", "very hard", "hard", "easy") fatta a priori sulla base della qualifica del ricevendo e del suo turno di prenotazione. E' possibile disdire una prenotazione fino a due giorni prima che il ricevimento cominci. Nel caso una prenotazione sia cancellata, lo *slot* diventa nuovamente disponibile. I dati delle prenotazioni sono conservati per un mese, dopodiché vanno cancellati.

Glossario

Il glossario ha lo scopo fondamentale di chiarire il gergo tecnico usato nella descrizione dei requisiti e di evidenziare eventuali sinonimie e omonimie.

In questo caso la terminologia è immediata e sono semplicemente evidenziati alcuni sinonimi.

TERMINE	DEFINIZIONE	SINONIMI	OMONIMI
studente	persona iscritta al ricevimento	ricevendo, discente, prenotato	—
slot	singolo incontro docente-studente	incontro, turno	—

Diagramma EE/R

Il diagramma EE/R è visibile in figura 1. Si fa notare che il vincolo che tutti i ricevendi abbiano prenotato almeno un ricevimento rimane inespresso poiché il docente carica i dati dei ricevendi prima che questi possano prenotarsi. Non sono indicate le cardinalità massime e minime delle associazioni.

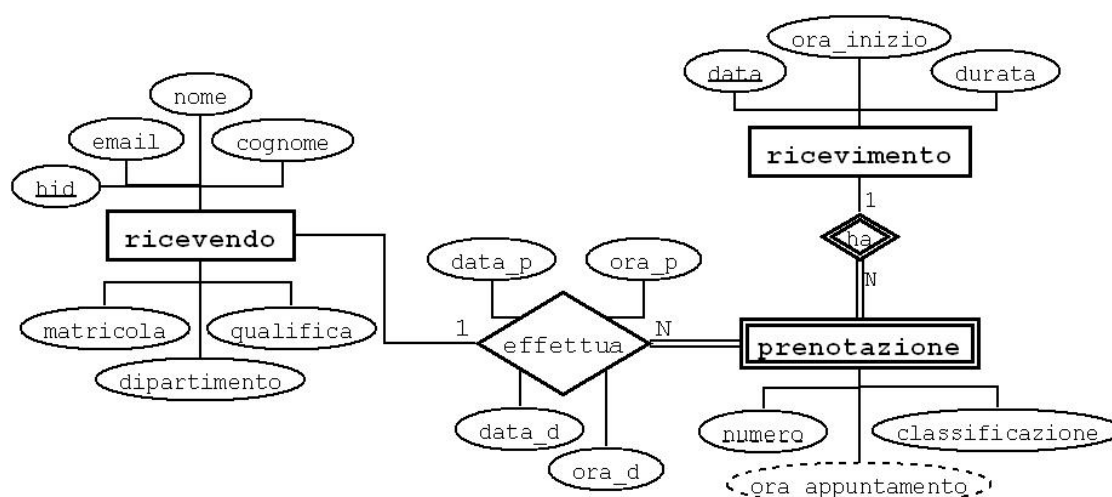


Figura 1: Diagramma Entità-Associazione

Diagramma relazionale

Il diagramma relazionale è visibile in figura 2. Si fa notare che tutte le coppie di attributi *data* e *ora* riferite allo stesso evento sono state tradotte in un campo unico, in vista dell'implementazione nel DBMS scelto.⁹ *ora_appuntamento* in quanto attributo derivato è assente dalla tabella *prenotazione*, mentre *classificazione* è un attributo il cui valore segue regole complesse e sarà determinato automaticamente tramite *trigger*.

⁹ In Oracle il tipo nativo DATE include anche l'ora.

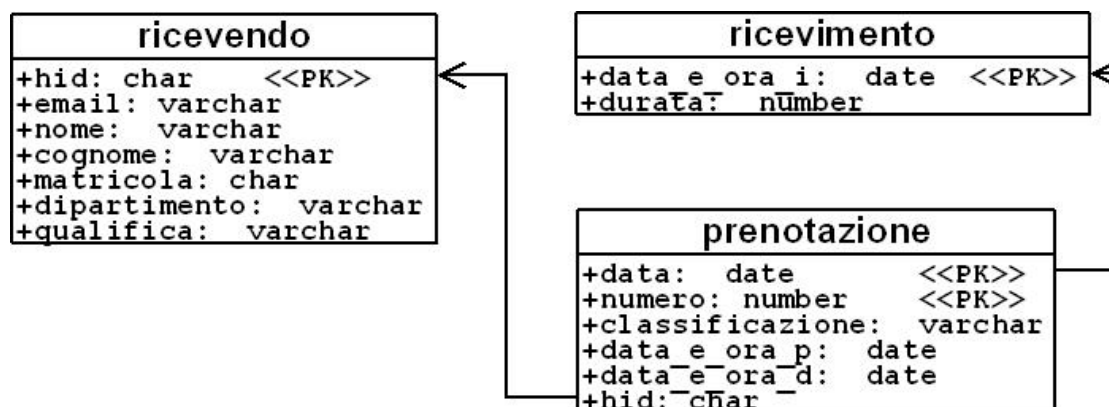


Figura 2: Diagramma relazionale

Utenti e loro categorie

La prenotazione di un ricevimento avviene attraverso un unico utente le cui credenziali sono pubbliche, e dunque il DB avrà solo tre utenti: un amministratore, un discente (account pubblico) e un docente (Monsieur De La Palisse, il committente). Dato lo scenario semplificato, non occorrono ruoli¹⁰ per gestire i permessi ed è sufficiente un uso moderato delle viste.

La tavola degli utenti è rappresentata nella tabella 1. Si fa notare che non sono indicati i privilegi di sistema, come il CONNECT, ma solo quelli di oggetto.

UTENTE	TIPO	VOLUME	PERMESSI
DB_prenotati	Amministratore	1	ALL
Docente	Comune	1	ALL ON ricevendo EXECUTE prog_ric SELECT ON prenotati
Discente	Comune	1	SELECT ON ric_disp EXECUTE prenota EXECUTE disdici

Tabella 1: Tavola degli utenti.

Operazioni degli utenti

Le operazioni di base sono quelle eseguibili con un semplice comando DML. In questo caso le uniche operazioni di base sono l'inserimento, l'aggiornamento o la modifica di una tupla di RICEVENDO o la visualizzazione dei prenotati con una vista da parte del docente, e la visualizzazione dei ricevimenti disponibili da parte del discente.

Le operazioni degli utenti sono quelle più complesse che vanno implementate tramite procedure, e in questo caso sono la prenotazione, la disdetta e la programmazione di un ricevimento. Nel diagramma dei casi d'uso in figura 3 sono visibili sia le operazioni implementate che gli utenti coinvolti.

¹⁰Un ruolo, *role* in inglese, è un gruppo denominato di permessi scelti dall'amministratore di sistema. È utile per gestire utenti con gli stessi privilegi.



Figura 3: Diagramma UML dei casi d'uso

Seguono le schede descrittore operazione per le operazioni degli utenti:

OPERAZIONE	prog_ric
SCOPO:	fissare l'ora e la data di un ricevimento
ARGOMENTI:	data e ora, durata
RISULTATO:	aggiunta ricevimento/errore
ERRORI:	giorno non consentito
USA:	ricevimento, prenotazione
MODIFICA:	ricevimento, prenotazione
PRIMA:	non c'è un altro ricevimento in quel giorno
POI:	c'è un solo ricevimento in quel giorno

OPERAZIONE	prenota
SCOPO:	prenotare un ricevimento
ARGOMENTI:	data del ricevimento, hid
RISULTATO:	aggiunta prenotazione/errore
ERRORI:	giorno senza ricevimento hid errato troppo tardi troppo presto ricevimento saturo ricevimento già passato
USA:	prenotazione
MODIFICA:	prenotazione
PRIMA:	c'è ricevimento e c'è posto in quel giorno
POI:	uno slot è assegnato al dato hid

OPERAZIONE	disdici
SCOPO:	disdire una prenotazione
ARGOMENTI:	data del ricevimento, hid
RISULTATO:	prenotazione disdetta/errore
	giorno senza prenotazione
ERRORI:	hid errato troppo tardi
USA:	prenotazione
MODIFICA:	prenotazione
PRIMA:	c'è una prenotazione in quel giorno
POI:	la data disdetta è inserita sullo slot

La tavola delle operazioni è rappresentata in tabella 2. Si fa notare che occorre contare il numero di volte in cui l'operazione è lanciata, non necessariamente il numero di volte in cui ha successo.

In questo caso si ipotizza che si tentino 40 prenotazioni a settimana, sebbene in una settimana di norma si tenga un solo ricevimento (quindi al massimo 20 persone) e molte prenotazioni falliscano.

OPERAZIONE	TIPO	VOLUME	PERIODO
Programmazione ricevimento	B	4	mese
Prenotazione ricevimento	B	40	settimana
Disdetta prenotazione	B	5	settimana

Tabella 2: Tavola delle operazioni. B sta per *batch* e I per *interactive*. Usando solo SQL statico, tutte le operazioni sono *batch*.

Volumi

La tavola dei volumi è rappresentata nella tabella 3. Oltre a riportare il numero verosimile di tuple presenti in ciascuna tabella una volta che il DB sia a regime, rappresenta anche il suo incremento atteso in un periodo di tempo prefissato. Se esiste una politica di aggiornamento dei dati per cui le tuple più vecchie sono rimosse dal DB, oppure se il numero di tuple in una relazione è più o meno costante nel tempo, allora l'incremento è nullo.

In questo caso si ipotizza che un discente nella tabella *ricevendo* non sia mai cancellato, mentre le tuple di *ricevimento* e *prenotazione* siano conservate per un anno.

TABELLA	TIPO	VOLUME	INCREMENTO	PERIODO
ricevendo	E	200	50	anno
ricevimento	E	4	0	mese
prenotazione	ED	80	0	mese

Tabella 3: Tavola dei volumi. E sta per *entità* e ED per *entità debole*. Per le tabelle di transizione si indica una A, che sta per *associazione*.

Vincoli di integrità

Sono detti statici i vincoli che limitano i valori assumibili da alcuni attributi indipendentemente dal tempo, mentre sono detti dinamici quelli che riguardano valori che cambiano nel tempo; anche molte *regole di business* si presentano come vincoli dinamici e la differenza con questi ultimi è molto sottile. Poiché tutti i dati sono soggetti ad evoluzione con velocità diverse, non si può essere attaccati alla lettera di queste definizioni, né si può pretendere di produrre un elenco esaustivo dei vincoli pertinenti ad un dato contesto.

Si riportano i vincoli più importanti per il DB *prenotati*. Non compaiono nell'elenco, in quanto ovvi, i vincoli di chiave primaria e chiave esterna.

Statici

- Per i ricevendi sono ammesse le seguenti qualifiche: dottorando, tesista, studente e altro.
- L'email di un ricevente è univoca e obbligatoria.
- Le prime quattro cifre della matricola determinano il dipartimento.
- Non è possibile fissare un ricevimento il sabato o la domenica.
- La durata di un ricevimento è compresa tra 30 e 200 minuti ed è un multiplo di 10.

Dinamici

- Un ricevimento non si può prenotare il giorno stesso in cui si tiene o con più di tre settimane di anticipo.
- Un ricevimento si disdice al più tardi con due giorni di anticipo.

Verifica di normalità

Nella verifica di normalità si combatte la ridondanza attraverso la caccia alle dipendenze funzionali anomale. Mentre il rispetto della prima forma normale è in qualche modo dato per scontato, implicito nel modello relazionale, la seconda e la terza vanno verificate analizzando il significato degli attributi, poiché si tratta di una proprietà intensionale, non estensionale. Non è obbligatorio portare il database in terza forma normale ed è opportuno tenere presente che ci sono casi in cui è preferibile per ragioni di prestazioni tenere una forma normale più bassa, in ogni caso le scelte operate devono essere motivate.

Prima forma normale

La cosa più ovvia, ovvero il rispetto della prima forma normale, necessita della spiegazione più dettagliata: a rigore il DB non è in prima forma normale, sia per la presenza di vari campi unici data e ora, sia perché il *numero* di matricola non è né un numero, né un valore atomico.¹¹

Il campo data è un tipo primitivo nella maggior parte dei linguaggi di programmazione, e pur essendo in realtà strutturato è convenzionalmente considerato atomico. Vale lo stesso per il campo data e ora di Oracle DBMS, a patto di compiere una forzatura più ampia dell'accezione di atomicità. La conseguenza indesiderata di questa scelta è che la chiave dell'entità debole

¹¹Non è un numero perché è un codice e non è atomico perché è composto da un codice di corso di laurea e un codice studente.

prenotazione include solo una parte dell'attributo *data_e_ora_i* e bisogna prevenire anomalie di inserimento.

Con *numero di matricola* d'altro canto si denomina solitamente la concatenazione del codice corso di laurea e dell'identificativo dello studente nel corso, sebbene queste siano informazioni separate, e mantenere l'accezione comune significa generare delle anomalie, come la dipendenza funzionale tra una parte del numero di matricola e il nome del dipartimento.¹²

Seconda forma normale

L'unica chiave multiattributo è quella di *prenotazione*, che non determina parzialmente alcun altro attributo della relazione. Quindi date per buone le nozioni di atomicità definite nel paragrafo precedente, la seconda forma normale è rispettata.

Terza forma normale e BCNF

Se la matricola fosse rappresentata con due campi, un codice corso di laurea (*CCDL*) e un codice studente (*CS*), allora la dipendenza $CCDL \rightarrow dipartimento$ sarebbe una dipendenza anomala e lo schema non sarebbe né in 3NF né in BCNF; poiché la matricola è rappresentata con un unico campo, la dipendenza $matricola \rightarrow dipartimento$ non esiste¹³ e lo schema è in BCNF.

Possibili estensioni

Le estensioni possibili sono innumerevoli e il limite è dato solo dalla fantasia: semplice è cambiare le eccezioni in modo che anziché fallire prenotino la prima data utile, sia per il docente che per il discente; utile è una funzione che mostra lo storico di tutte le prenotazioni di una *email* o di un *hid*.

Vincoli dinamici/*regole di business* alternativi a quelli dati, ognuno dei quali comporta modifiche più o meno estese al codice proposto, sono:

- Se il discente si è già prenotato per uno dei prossimi ricevimenti e non ha disdetto, non può fare un'altra prenotazione.
- Una volta disdetto un ricevimento non è più possibile prenotarlo lo stesso giorno.
- Si può prenotare solo per la settimana corrente, dal mercoledì al martedì successivo, ma non oltre.
- Una persona è ricevuta al massimo tre volte al mese.
- Chi ha disdetto più di tre volte in un mese non può più prenotarsi in quel mese.
- Gli utenti che disdicono più di tre ricevimenti consecutivi sono cancellati dalla lista dei ricevendi.

Per quanto concerne lo *scheduler*, si può pensare ad un *job* che ogni lunedì mattina avvii la procedura di creazione ricevimento con un orario standard per la settimana corrente, oppure che ogni primo del mese programmi i ricevimenti per tutto il mese a venire, escludendo di fatto il docente da qualunque programmazione del ricevimento (si presuppone che il docente sia sempre disponibile). E' altresì possibile implementare politiche sanzionatorie o

¹²Attenzione, questa non è una dipendenza funzionale parziale secondo la teoria della normalizzazione.

¹³La dipendenza tra il prefisso della matricola e *dipartimento* è controllata tramite un vincolo di tupla.

premiali tramite *job*, ad esempio: “i discenti che non prenotano ricevimento per più di tre mesi sono cancellati dalla lista dei ricevendi”; “ogni mese il(i) ricevendo(i) più assiduo(i) è(sono) prenotato(i) automaticamente al primo(secondo/terzo/...) posto nel prossimo ricevimento”.

Volendo conservare l'impianto del codice così com'è, un'estensione interessante sarebbe usare il *package* UTL_MAIL nel corpo del trigger *email2hid* per mandare automaticamente una email al ricevendo con il suo *hid*, oppure per mandare conferma al docente e all'amministratore di tutte le operazioni compiute dai vari *job*.

Volendo stravolgere l'impianto del codice, è possibile fare in modo che siano le richieste che si accumulano a determinare i giorni di ricevimento: ogni giorno un *job* controlla il numero di prenotati, e se è raggiunta una soglia prefissata allora si programma un ricevimento, magari mandando una email di conferma al docente e a tutti i prenotati. Si può pensare ad una coda con priorità per gestire le prenotazioni, che tenga conto della qualifica e/o dell'assiduità dei ricevendi, intrecciata a varie regole di alternanza (per esempio ogni due tesisti c'è necessariamente uno studente *et similia*). E' possibile pensare a *slot* di rata variabile e stimare la durata presunta di ogni appuntamento, il numero di appuntamenti e l'orario con una variabile casuale di *Poisson*, prevedendo una percentuale di *overbooking* e inserendo una penalità per i ritardatari o gli assenti. Complicare è facile, semplificare è difficile.

La gestione della banda, della memoria, gli aspetti relativi alla sicurezza, alla privacy e alla concorrenza, le politiche di *backup*, l'ottimizzazione di performance sono stati appena sfiorati o deliberatamente ignorati. Ognuno di questi richiederebbe competenze a sé e non può essere incluso in un corso di primo livello.

Implementazione

Utenti
DDL
DML
Trigger
Procedure
Viste
DCL
Scheduler

In questo capitolo si tratta l'implementazione del DB. Tutto quanto documentato nel capitolo precedente è tradotto in codice eseguibile sul DBMS Oracle 10 Express Edition.

Implementazione

“I *bug* più sottili causano i danni e i problemi più grandi.”

COMPLETATA LA PROGETTAZIONE, le specifiche devono essere convertite in codice eseguibile, ed è quanto illustrato in questo capitolo. Il codice si riferisce al DBMS Oracle 10g XE e il linguaggio adottato è il PL/SQL. L'ordine con cui lanciare gli script non è necessariamente quello con cui sono presentati nel capitolo, che riflette piuttosto l'organizzazione logica dell'argomento. Tutti i nomi arbitrari sono in minuscolo, mentre le parole riservate in maiuscolo, opportunamente evidenziate in blu.

Creazione utenti

Il primo passo da compiere è accedere al DBMS come amministratore di sistema e creare l'utente proprietario della base di dati. E' possibile creare contestualmente anche altri utenti, sebbene essi non possano ricevere alcun privilegio di oggetto, poiché lo schema ancora non esiste.

```
CREATE USER db_prenotati IDENTIFIED BY admin;  
CREATE USER docente IDENTIFIED BY la_palisse;  
CREATE USER discente IDENTIFIED BY supereroe;  
GRANT ALL PRIVILEGES TO db_prenotati;
```

In seguito occorre disconnettersi e riconnettersi con le credenziali dell'utente *db_prenotati*, che è l'amministratore della nuova base di dati e che avrà i permessi per creare tutti gli oggetti che occorrono.¹⁴

Data Definition Language

Il DDL riflette pedissequamente lo schema relazionale: le tre tabelle sono create mediante altrettante istruzioni di CREATE TABLE che includono tutti i vincoli di integrità esprimibili

¹⁴*db_prenotati* ha ricevuto tutti i privilegi ed è quindi amministratore dell'intero DBMS. Nel caso il DBMS ospiti più DB, è opportuno limitare i privilegi di ciascun amministratore alla sola base di dati di cui è proprietario e usare l'account SYSTEM come amministratore di sistema.

nel modello relazionale (l'obbligatorietà, i vincoli di dominio statico, l'univocità, l'integrità referenziale, etc.). Va sottolineato che in Oracle il tipo DATE include l'ora, e dunque i campi di tipo data e ora (con tutte le loro varianti come i campi ora e minuto, minuto e secondo etc.), pur se separati nel diagramma E/R, sono tradotti con un campo unico di tipo DATE.

Ricevendo

La tabella *ricevendo* è popolata dal docente tramite inserimento diretto, dunque è necessario includere tutti i controlli di integrità possibili nel DDL.

Si noti il vincolo di tupla sul prefisso della matricola, che determina il nome del dipartimento. E' possibile definire espressioni regolari anche per il nome e il cognome, come per quasi tutti i campi di tipo carattere.

```
--%%%%%%%%%          TABELLA RICEVENDO
CREATE TABLE ricevendo (
  hid                CHAR(6)                PRIMARY KEY,
  email              VARCHAR2(30)           UNIQUE NOT NULL,
  nome               VARCHAR2(15)           NOT NULL,
  cognome            VARCHAR2(15)           NOT NULL,
  matricola          CHAR(9)                UNIQUE,
  dipartimento       VARCHAR2(15),
  qualifica          VARCHAR2(10),
  CONSTRAINT qualifiche_ammesse CHECK(
    LOWER(qualifica) IN
      ('dottorando','tesista','studente','altro')),
  CONSTRAINT matr_ammesse CHECK(
    SUBSTR(matricola,1,4) IN
      ('0123','0124','0108')),
  CONSTRAINT dipartimenti_ammessi CHECK(
    INITCAP(dipartimento) IN
      ('Mitolalia','Fantascienze','Xenologia')),
  CONSTRAINT codici_dip CHECK(
    (SUBSTR(matricola,1,4)='0123' AND
      INITCAP(dipartimento)='Mitolalia')
    OR
    (SUBSTR(matricola,1,4)='0124' AND
      INITCAP(dipartimento)='Fantascienze')
    OR
    (SUBSTR(matricola,1,4)='0108' AND
      INITCAP(dipartimento)='Xenologia')),
  CONSTRAINT email_mask CHECK(
    REGEXP_LIKE(email,'^\w+.*@{1}\w+.*$')),
  CONSTRAINT matricola_mask CHECK(
    REGEXP_LIKE(matricola,'^d{4}[/]d{4}$'))
);
```

Ricevimento

La tabella *ricevimento* è popolata tramite la procedura *prog_ric*, lanciata dal docente, e la maggior parte dei vincoli di integrità è gestita all'interno della procedura stessa, rendendone superflua la definizione nel DDL. Tale approccio permette una maggiore flessibilità nella gestione dei dati.

Si evidenzia una particolarità: il campo *data_e_ora_i* è la concatenazione di due campi del diagramma E/R di cui uno è la chiave primaria, e dichiararlo *tout court* PRIMARY KEY

permetterebbe di fissare più ricevimenti nello stesso giorno in orari diversi, in contraddizione con i requisiti del sistema. Si potrebbe pensare ad una funzione che estrae solo la data dal campo data e ora, ma non è consentito definire il vincolo di chiave su una funzione di un campo. La soluzione è definire un ulteriore vincolo di univocità, esterno alla tabella, che garantisca le date univoche. Questo vincolo si aggiunge a quello di chiave primaria, in modo che in fase di popolamento, perché una tupla sia legale, debba soddisfare entrambe le univocità: quella della data con l'ora inclusa e quella della data in sé.¹⁵

E' espresso un vincolo di dominio per imporre che il ricevimento si tenga in orari d'ufficio, mentre i vincoli sulla sua durata e sull'esclusione dei giorni festivi dalle date utili sono inclusi nella procedura di popolamento.

```
--%%%%%%%%%          TABELLA RICEVIMENTO
CREATE TABLE ricevimento
(
  data_e_ora_i      DATE          PRIMARY KEY,
  durata            NUMBER(3)     /* in minuti */,
  CONSTRAINT ore_ufficio CHECK
    (TO_CHAR(data_e_ora_i,'HH24') BETWEEN 8 AND 17)
);

CREATE UNIQUE INDEX data_univoca
  ON ricevimento (TRUNC(data_e_ora_i));
```

Prenotazione

La tabella *prenotazione* è popolata tramite la procedura *prenota*, che include i controlli sulle date legittime e sui posti disponibili.

Il controllo di univocità sulla coppia (*data,email*) che impedisce ad uno studente di prenotarsi più volte per lo stesso giorno potrebbe essere eseguito con una dichiarazione UNIQUE (*data,email*) alla fine della tabella, ma dato che la prenotazione avviene tramite una procedura che crea prima gli *slot* da assegnare senza alcuna email associata, il controllo di univocità coinvolgerebbe dei valori mancanti. In linea teorica ciò non comporta differenze o problemi, ma poiché il DBMS Oracle non gestisce in maniera semanticamente corretta i valori mancanti (in questo caso è come se li considerasse uguali), è necessario definire un vincolo di univocità esterno con una funzione CASE che trasforma un singolo valore mancante in una coppia (NULL,NULL), per la quale sono ammessi i duplicati.

Si noti il vincolo di tupla *fwd_time*, che vuole la disdetta successiva alla prenotazione.

```
--%%%%%%%%%          TABELLA PRENOTAZIONE
CREATE TABLE prenotazione
(
  data              DATE,
  numero            NUMBER(2) /*inutile CHECK (numero<21)*/,
  classificazione   VARCHAR2(11)
    CHECK (LOWER(classificazione) IN
      ('easy','not so easy','uneasy','hard','very hard')),
  data_e_ora_p      DATE,
  data_e_ora_d      DATE,
  hid               CHAR(6) /*rendendolo obbligatorio
    si andrebbe in contrasto con la procedura prenota*/,
  CONSTRAINT pkp    PRIMARY KEY (data,numero),
  CONSTRAINT fkrn   FOREIGN KEY (data)
    REFERENCES ricevimento(data_e_ora_i) ON DELETE CASCADE,
```

¹⁵ Si noti che se si usasse solo il secondo vincolo, si garantirebbe sì l'univocità delle date, ma non si potrebbe usare *data_e_ora_i* come chiave esterna.

```

CONSTRAINT fkrv FOREIGN KEY (hid)
    REFERENCES ricevendo(hid) ON DELETE SET NULL,
CONSTRAINT fwd_time CHECK (data_e_ora_d>data_e_ora_p)
/*la disdetta deve essere successiva alla prenotazione*/
);

CREATE UNIQUE INDEX no_stalking ON prenotazione
(CASE WHEN data IS NULL OR hid IS NULL THEN NULL ELSE data END,
 CASE WHEN hid IS NULL OR data IS NULL THEN NULL ELSE hid END);

```

Data Manipulation Language

Il popolamento di *ricevimento* e di *prenotazione* avviene attraverso le apposite procedure, che lo facilitano e garantiscono i controlli di integrità sui dati. L'unica tabella in cui il popolamento avviene attraverso delle operazioni dirette di INSERT è *ricevendo* e l'unico utente abilitato ad eseguirle è *docente*.

```

--%%%%%%%% AGGIUNTA DI UN RICEVENDO
INSERT INTO ricevendo VALUES(NULL,'ironman@parthenope.it','Anthony',
    'Stark','0124/0001','Fantascienze','tesista');
INSERT INTO ricevendo VALUES(NULL,'superman@parthenope.it','Clark',
    'Kent','0108/0005','Xenologia','studente');
INSERT INTO ricevendo VALUES(NULL,'wonderwoman@parthenope.it','Diana',
    'Prince','0123/0007','Mitolalia','studente');
INSERT INTO ricevendo VALUES(NULL,'batman@parthenope.it','Buce','Wayne',
    '0123/0043','Mitolalia','tesista');
INSERT INTO ricevendo VALUES(NULL,'robin@parthenope.it','Dick','Grayson',
    '0123/0664','Mitolalia','studente');
INSERT INTO ricevendo VALUES(NULL,'hulk@parthenope.it','David','Banner',
    '0124/0011','Fantascienze','tesista');
INSERT INTO ricevendo VALUES(NULL,'daredevil@parthenope.it','Matt','Murdock',
    '0124/0061','Fantascienze','tesista');
INSERT INTO ricevendo VALUES(NULL,'spiderman@parthenope.it','Peter','Parker',
    '0124/0033','Fantascienze','studente');
INSERT INTO ricevendo VALUES(NULL,'goblin@parthenope.it','Norman','Osborn',
    '0124/0074','Fantascienze','studente');
INSERT INTO ricevendo VALUES(NULL,'droctopus@parthenope.it','Otto','Octavius',
    '0124/0567','Fantascienze','studente');
INSERT INTO ricevendo VALUES(NULL,'lizard@parthenope.it','Curtis','Connors',
    '0124/0099','Fantascienze','tesista');
INSERT INTO ricevendo VALUES(NULL,'wolverine@parthenope.it','James','Howlett',
    '0124/0088','Fantascienze','studente');
INSERT INTO ricevendo VALUES(NULL,'magneto@parthenope.it','Max','Eisenhard',
    '0108/0029','Xenologia','dottorando');
INSERT INTO ricevendo VALUES(NULL,'mrfantastic@parthenope.it','Reed','Richards',
    '0124/0021','Fantascienze','dottorando');
INSERT INTO ricevendo VALUES(NULL,'thing@parthenope.it','Ben','Grimm',
    '0123/0013','Mitolalia','studente');
INSERT INTO ricevendo VALUES(NULL,'humantorch@parthenope.it','Johnny','Storm',
    '0108/0325','Xenologia','studente');
INSERT INTO ricevendo VALUES(NULL,'invisiblewoman@parthenope.it','Susan','Storm',
    '0108/0114','Xenologia','tesista');
INSERT INTO ricevendo VALUES(NULL,'drdoom@uniparthenope.it','Victor','Von Doom',
    '0108/0432','Xenologia','altro');

```


A titolo di mero esempio sono mostrate anche le operazioni di INSERT su *ricevimento* e di INSERT e UPDATE su *prenotazione*, poiché solo l'amministratore potrebbe eseguirle, eludendo i controlli presenti nelle rispettive procedure.

```
--%%%%%%%%%      AGGIUNTA DI UN RICEVIMENTO
INSERT INTO ricevimento VALUES (TO_DATE('6-mag-2014
14:30','DD-MON-YYYY HH24:MI'),120);
INSERT INTO ricevimento VALUES (TO_DATE('13-mag-2014
14:30','DD-MON-YYYY HH24:MI'),200);
INSERT INTO ricevimento VALUES (TO_DATE('20-mag-2014
14:30','DD-MON-YYYY HH24:MI'),180);
INSERT INTO ricevimento VALUES (TO_DATE('27-mag-2014
14:30','DD-MON-YYYY HH24:MI'),180);
INSERT INTO ricevimento VALUES (TO_DATE('3-giu-2014
14:30','DD-MON-YYYY HH24:MI'),120);
INSERT INTO ricevimento VALUES (TO_DATE('10-giu-2014
14:30','DD-MON-YYYY HH24:MI'),200);
INSERT INTO ricevimento VALUES (TO_DATE('17-giu-2014
14:30','DD-MON-YYYY HH24:MI'),120);
INSERT INTO ricevimento VALUES (TO_DATE('24-giu-2014
14:30','DD-MON-YYYY HH24:MI'),180);
INSERT INTO ricevimento VALUES (TO_DATE('1-lug-2014
14:30','DD-MON-YYYY HH24:MI'),120);
INSERT INTO ricevimento VALUES (TO_DATE('3-lug-2014
12:30','DD-MON-YYYY HH24:MI'),180);
INSERT INTO ricevimento VALUES (TO_DATE('22-lug-2014
14:30','DD-MON-YYYY HH24:MI'),180);

--%%%%%%%%%      AGGIUNTA DI UNA PRENOTAZIONE
INSERT INTO prenotazione VALUES (
TO_DATE('6-mag-2014 14:30','dd-mon-yyyy HH24:MI'),1,'easy',
TO_DATE('1-mag-2014 13:27','dd-mon-yyyy HH24:MI'),NULL,777256);

INSERT INTO prenotazione VALUES (
TO_DATE('6-mag-2014 14:30','dd-mon-yyyy HH24:MI'),2,'uneasy',
TO_DATE('2-mag-2014 16:31','dd-mon-yyyy HH24:MI'),NULL,241285);

INSERT INTO prenotazione VALUES (
TO_DATE('6-mag-2014 14:30','dd-mon-yyyy HH24:MI'),3,'hard',
TO_DATE('4-mag-2014 08:01','dd-mon-yyyy HH24:MI'),NULL,583089);

INSERT INTO prenotazione VALUES (
TO_DATE('6-mag-2014 14:30','dd-mon-yyyy HH24:MI'),4,'hard',
TO_DATE('2-mag-2014 05:50','dd-mon-yyyy HH24:MI'),NULL,668628);

INSERT INTO prenotazione VALUES (TO_DATE('6-mag-2014 14:30','dd-
mon-yyyy HH24:MI'),5,NULL,NULL,NULL);

--%%%%%%%%%      DISDETTA DI UNA PRENOTAZIONE
UPDATE prenotazione
SET data_e_ora_d=TO_DATE('3-mag-2014 13:31','dd-mon-yyyy HH24:MI')
WHERE data=TO_DATE('6-mag-2014 14:30','dd-mon-yyyy HH24:MI')
AND hid=668628;
```

Trigger

I trigger DML sono utili principalmente per il controllo di vincoli dinamici in fase di immisione o aggiornamento dei dati. E' possibile *cum grano salis* usarli anche per implementare regole di business o per calcolare, generare o sovrascrivere il valore di alcuni campi.

Siccome in questo caso tutti i vincoli dinamici fondamentali sono controllati tramite le procedure di popolamento o il DDL, i trigger sono usati per popolare i campi *hid* e *classificazione*.

Hashing

Usare l'email come chiave primaria di ricevendo non è una buona idea, poiché è un dato facilmente reperibile (supponiamo si usino indirizzi istituzionali) e la conoscenza dell'email di una persona permetterebbe ad un'altra di prenotare o di disdire in sua vece. Lo stesso varrebbe se si utilizzasse come ID un semplice contatore, facile da indovinare, ed è quindi più sicuro applicare una funzione *hash* al campo *email* in modo da generare come ID un numero pseudo-casuale, che il ricevendo sarà l'unico a conoscere e poter utilizzare.

Il trigger seguente calcola automaticamente il valore del campo *hid* all'atto dell'inserimento di un ricevendo. Nella tupla di INSERT il campo *hid* va lasciato vuoto, sarà il trigger a calcolarlo e popolarlo automaticamente. Dato il ridotto numero di indirizzi email coinvolti, la probabilità di una collisione (generare lo stesso valore per indirizzi email diversi) è praticamente nulla. Le collisioni non sono gestite.

```
--%%%%%   CALCOLA AUTOMATICAMENTE L'ID DEL RICEVENDO
CREATE OR REPLACE TRIGGER email2hid
BEFORE INSERT ON ricevendo
FOR EACH ROW
BEGIN
-- calcolo e assegno il valore della funzione hash
SELECT ORA_HASH(LOWER(:NEW.email),999999) INTO :NEW.hid FROM DUAL;
-- stampo a video il valore di hid
DBMS_OUTPUT.PUT_LINE('L''hash ID di '||:NEW.email|| ' e'' '|| :NEW
.hid);
END;
```

Classificazione

Il trigger seguente imposta automaticamente il valore del campo *classificazione*, che rappresenta la difficoltà presunta di ogni appuntamento, sulla base della qualifica del discente e del suo numero di prenotazione.¹⁶ Si assume che la difficoltà sia direttamente proporzionale al livello di istruzione del ricevendo e al tempo trascorso dall'inizio della sessione (col passare del tempo il docente perde lucidità). Stabilito un sistema di punteggio per cui un dottorando vale 4 punti, un tesista 3 punti, uno studente 2 punti e gli altri 1; stabilito che i primi 4 appuntamenti valgono 1 punto, i successivi 4 valgono 2 punti e così via; il punteggio finale sarà il prodotto dei due punteggi precedenti convertito in scala ordinale secondo la Tabella 4. Il sistema dei punteggi è implementabile attraverso l'uso della funzione DECODE, che evita IF multipli o istruzioni CASE.

```
--%%%%%   IMPOSTA AUTOMATICAMENTE IL LIVELLO DI DIFFICOLTA'
CREATE OR REPLACE TRIGGER pop_classificazione
```

¹⁶ Si noti che una regola semplice come “se la qualifica è dottorando, allora il tipo di ricevimento è hard; se è studente, allora è easy”, genererebbe una dipendenza funzionale anomala e comporterebbe che *classificazione* sia un attributo di *ricevendo*.

PUNTI	CLASSIFICAZIONE
$P \leq 4$	easy
$4 < P \leq 8$	not so easy
$8 < P \leq 12$	uneasy
$12 < P \leq 16$	hard
$16 < P$	very hard

Tabella 4: Scala ordinale dei livelli di difficoltà.

```

BEFORE UPDATE OF hid ON prenotazione
FOR EACH ROW
DECLARE
punti_qualifica    NUMBER(1);
punti_turno        NUMBER(1);
BEGIN
-- calcolo i punti per la qualifica del ricevendo
SELECT DECODE(LOWER(qualifica), 'dottorando', 4, 'tesista', 3, '
    studente', 2, 'altro', 1) INTO punti_qualifica
    FROM ricevendo
    WHERE hid=:NEW.hid;
-- calcolo i punti per il turno
SELECT CEIL(:NEW.numero/4) INTO punti_turno FROM DUAL;
-- assegno il livello di difficoltà
SELECT DECODE(CEIL(punti_qualifica*punti_turno/4), 1, 'easy', 2, 'not
    so easy', 3, 'uneasy', 4, 'hard', 5, 'very hard')
    INTO :NEW.classificazione FROM DUAL;
EXCEPTION
WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20001, 'hid insistente');
END;

```

Procedure e funzioni

Le procedure sono la parte più strettamente legata alla *logica di business* e all'automazione della Base di Dati. Se nel DDL e nel DML il problema principale è decidere la rappresentazione ottima dei dati e garantire rispetto dei vincoli di integrità su di essi definiti, in questa fase il problema è come sfruttare al meglio tale rappresentazione per automatizzarne la gestione e trarne il massimo profitto.

Programmazione ricevimento

Il *docente* fissa un ricevimento lanciando la procedura *prog_ric*, che ha in input il giorno e la durata previsti (le informazioni strettamente necessarie). In base alla durata, la procedura crea tanti *slot* di dieci minuti sulla tabella *prenotazione* fino a coprire l'intera sessione di ricevimento, che diventano immediatamente prenotabili tramite un UPDATE.¹⁷

```

--%%%% PROGRAMMAZIONE RICEVIMENTO, lanciabile dall'utente
DOCENTE
CREATE OR REPLACE PROCEDURE prog_ric
    (data DATE, durata_in ricevimento.durata%TYPE)

```

¹⁷Ciò è preferibile in termini di gestione della concorrenza a permettere la prenotazione con un INSERT diretto sulla tabella *prenotazione*.

```

IS
--DECLARE
durata          NUMBER :=durata_in;
sabato_fascista EXCEPTION;
BEGIN
IF TO_CHAR(data,'D')=6 OR TO_CHAR(data,'D')=7 THEN
    RAISE    sabato_fascista;
END IF;
IF durata<30 THEN
    DBMS_OUTPUT.PUT_LINE('Il ricevimento dura minimo 30 minuti');
    durata:=30;
ELSIF durata>200 THEN
    DBMS_OUTPUT.PUT_LINE('Il ricevimento dura massimo 200 minuti');
    durata:=200;
ELSE
    durata:=ROUND(durata/10)*10;
    DBMS_OUTPUT.PUT_LINE('La durata prevista e'' di '|| durata ||'
        minuti');
END IF;
INSERT INTO ricevimento VALUES(data,durata);
FOR i IN 1..(durata/10) LOOP
    INSERT INTO prenotazione VALUES(data,i,NULL,NULL,NULL,NULL);
END LOOP;
/*il ciclo FOR si puo' sostituire con un unico INSERT usando una
    query gerarchica*/
EXCEPTION
WHEN sabato_fascista THEN
    RAISE_APPLICATION_ERROR(-20002,'Non e'' possibile fissare i
        ricevimenti il sabato o la domenica.');
```

Prenotazione

L'utente *discente*, che è unico per tutti e le cui credenziali sono pubbliche, prenota un ricevimento lanciando la procedura di *prenotazione*, che ha in input la data e *lhid* del ricevente. Prima di lanciare la procedura il discente consulta la vista *ric_disp*, la quale fornisce i giorni utili ed il numero di posti disponibili.

```

--%%%%%    PRENOTAZIONE RICEVIMENTO, lanciata dall'utente DISCENTE
CREATE OR REPLACE PROCEDURE prenota (mio_ricevimento DATE, mio_hid
    ricevendo_hid%TYPE)
IS
--DECLARE
mia_data          DATE;
mio_numero        NUMBER(2);
ora_appuntamento DATE;
troppo_tardi      EXCEPTION;
troppo_presto     EXCEPTION;
tempus_fugit      EXCEPTION;
BEGIN
-- controllo che nel giorno scelto ci sia ricevimento
SELECT data_e_ora_i INTO mia_data
    FROM ricevimento
    WHERE TRUNC(data_e_ora_i)=TRUNC(mio_ricevimento);
-- controllo che la prenotazione avvenga con al massimo tre
    settimane e minimo un giorno di anticipo
```

```

IF TRUNC(mia_data)=TRUNC(SYSDATE) THEN
    RAISE troppo_tardi;
ELSIF mia_data<SYSDATE THEN
    RAISE tempus_fugit;
ELSIF mia_data>TRUNC(SYSDATE)+21 THEN
    RAISE troppo_presto;
END IF;
-- Seleziono il primo slot disponibile e lo blocco, in modo da
  evitare che venga prenotato da altri
SELECT numero INTO mio_numero
  FROM prenotazione
 WHERE data = mia_data
 AND hid IS NULL
 AND ROWNUM = 1
 ORDER BY numero
FOR UPDATE WAIT 30; /*questa clausola serve a mettere un lock
  sulla riga*/
-- prenoto lo slot bloccato in precedenza
UPDATE prenotazione SET data_e_ora_p=SYSDATE, hid=mio_hid
  WHERE data = mia_data AND numero = mio_numero;
-- calcolo l'ora dell'appuntamento
ora_appuntamento:=mia_data+1/144*(mio_numero-1);
-- confermo
COMMIT;
-- stampo a video la conferma e l' ora dell'appuntamento
DBMS_OUTPUT.PUT_LINE('L'appuntamento e' confermato. Presentati
  alle ore: ' || TO_CHAR(ora_appuntamento,'HH24:MI'));

EXCEPTION
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Nel giorno che hai scelto non c''e''
  ricevimento oppure e'' gia'' saturo. Scegli un altro giorno.');
```

```

WHEN troppo_tardi THEN
    DBMS_OUTPUT.PUT_LINE('Occorre almeno un giorno di anticipo per
  prenotarsi. Scegli un altro giorno.');
```

```

WHEN tempus_fugit THEN
    DBMS_OUTPUT.PUT_LINE('Non e'' possibile prenotarsi per
  ricevimenti che si sono gia'' tenuti.Scegli un altro giorno.');
```

```

WHEN troppo_presto THEN
    DBMS_OUTPUT.PUT_LINE('Ci si puo'' prenotare con al massimo tre
  settimane di anticipo. Scegli un altro giorno.');
```

```

END;
```

Disdetta

Per disdire un appuntamento occorre un'operazione di UPDATE sulla tabella delle prenotazioni, che inserisca il valore SYSDATE nel campo relativo alla data e ora di disdetta (*data_e_ora_d*). Siccome è preferibile non dare un accesso in UPDATE sulla tabella *prenotazione* al discente, si crea una procedura di disdetta.

Nel momento in cui un ricevedo già prenotato disdice un appuntamento, lo *slot* corrispondente diventa nuovamente disponibile e chiunque cerchi di prenotarsi deve prioritariamente sfruttare questo. Per implementare tale *regola di business* occorre modificare la procedura di prenotazione in modo da bloccare prima gli *slot* disdetti e poi quelli vacanti, sovrascrivendo le prenotazioni disdette: in tal modo un appuntamento mancato non comporta

la rimodulazione dell'orario di tutti gli appuntamenti successivi. La modifica alla procedura di prenotazione è lasciata come esercizio.

```
--%%%%%%%% DISDETTA RICEVIMENTO, lanciata dall'utente DISCENTE
CREATE OR REPLACE PROCEDURE disdici (mio_ricevimento DATE, mio_hid
    ricevendo_hid%TYPE)
IS
--DECLARE
mio_numero          NUMBER(2);
troppo_tardi        EXCEPTION;
BEGIN
-- controllo che il ricevendo sia effettivamente prenotato
SELECT numero INTO mio_numero
    FROM prenotazione
    WHERE TRUNC(data)=TRUNC(mio_ricevimento)
    AND hid=mio_hid;
-- controllo che la disdetta avvenga al piu' tardi con due giorni
  di anticipo
IF TRUNC(mio_ricevimento-3)<TRUNC(SYSDATE) THEN
    RAISE troppo_tardi;
END IF;
-- Disdico
UPDATE prenotazione SET data_e_ora_d=SYSDATE
    WHERE TRUNC(data) = TRUNC(mio_ricevimento)
    AND numero = mio_numero;
-- confermo
COMMIT;
-- stampo a video la conferma
DBMS_OUTPUT.PUT_LINE('L'appuntamento e' disdetto.');
```

```
EXCEPTION
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Nel giorno che hai scelto non hai un
prenotazione.');
```

```
WHEN troppo_tardi THEN
    DBMS_OUTPUT.PUT_LINE('Non puoi disdire con meno di due giorni
di anticipo.');
```

```
END;
```

Conteggio dei posti disponibili

Il conteggio dei posti avviene tramite una funzione che prende in input la data del ricevimento e restituisce il numero di posti liberi. Tale funzione è usata nella vista *ric_disp*, ed è utile anche per mostrare tramite semplici query SQL quale sia il numero di prenotati in tutti i ricevimenti che rispettano un condizioni date.

```
--%%%%%%%% CONTEGGIO DEI POSTI DISPONIBILI
CREATE OR REPLACE FUNCTION conta_slot (data_ricevimento DATE)
RETURN NUMBER
IS
--DECLARE
data_r          DATE;
posti_disp      NUMBER(2);
posti_occ       NUMBER(2);
BEGIN
-- controllo che nel giorno scelto ci sia ricevimento
```

```

SELECT data_e_ora_i INTO data_r
      FROM ricevimento
      WHERE TRUNC(data_e_ora_i)=TRUNC(data_ricevimento);
-- conto i posti disponibili e occupati
SELECT MAX(numero)-COUNT(hid)+COUNT(data_e_ora_d),
      COUNT(hid)-COUNT(data_e_ora_d)
      INTO posti_disp, posti_occ
      FROM prenotazione
      WHERE TRUNC(data)=TRUNC(data_r);
-- restituisco il numero di posti disponibili
RETURN posti_disp;
EXCEPTION
WHEN NO_DATA_FOUND THEN
  RETURN NULL;
END;

```

Viste

Le viste sono relazioni virtuali utili a regolamentare l'accesso ed aumentare la fruibilità dei dati per i vari utenti. E' possibile (e auspicabile) usare le viste per mostrare porzioni dei dati (i dati più recenti, meno recenti, quelli di una determinata area etc.), calcolare gli attributi derivati e applicare funzioni più o meno complesse.

Ricevimenti disponibili

La prima vista è utile all'utente *discente* e permette di visualizzare i ricevimenti prenotabili, cioè quelli successivi al momento in cui la vista stessa è aperta ed entro tre settimane da questo per i quali c'è ancora posto (almeno uno slot ha email mancante oppure data disdetta presente). Accanto alle date prenotabili compare il numero di posti disponibili, calcolato con la funzione *conta_slot*. Si ricorda che non è possibile prenotare nel giorno stesso in cui si tiene ricevimento.

```

--%%%%%% VISTA DEI RICEVIMENTI DISPONIBILI
CREATE OR REPLACE VIEW ric_disp AS
SELECT DISTINCT data_e_ora_i AS data,
      conta_slot(data_e_ora_i) AS disp
      FROM ricevimento LEFT OUTER JOIN prenotazione
      ON data_e_ora_i=data
      WHERE TRUNC(data_e_ora_i)>TRUNC(SYSDATE)
      AND TRUNC(data_e_ora_i)<TRUNC(SYSDATE)+21
      ORDER BY data_e_ora_i;

```

Prenotati

La seconda vista è utile all'utente *docente* e mostra analiticamente i prenotati per il prossimo ricevimento, se esiste, cioè il primo successivo al momento in cui è aperta. Essa mostra anche il campo derivato *ora_appuntamento*, calcolabile con una proiezione generalizzata¹⁸ e un cambio di formato della data.

```

--%%%%%% VISTA DEI PRENOTATI PER IL PROSSIMO RICEVIMENTO
CREATE OR REPLACE VIEW prenotati AS

```

¹⁸ Nel DBMS Oracle un numero frazionario sommato ad una data comporta una variazione proporzionale dell'ora associata alla data stessa: un minuto è 1/1440 di un giorno e 10 minuti 1/144.

```

SELECT hid,email, data,
       TO_CHAR(data+(numero-1)*1/144,'HH24:MI') AS ora,
       classificazione AS class
FROM ricevendo rdo NATURAL JOIN prenotazione pre
WHERE pre.data_e_ora_d IS NULL
AND pre.data>SYSDATE
AND NOT EXISTS (
  SELECT * FROM ricevimento rto
  WHERE rto.data_e_ora_i <pre.data
  AND rto.data_e_ora_i >SYSDATE);

```

Data Control Language

L'utente *discente* può solo visualizzare i ricevimenti disponibili e prenotarsi o disdire, attraverso le rispettive vista e procedure. Nonostante siano operazioni abbastanza sicure (il ricevendo è l'unico a conoscere il suo *hid*), si vuole impedire del tutto che *discente* acceda alle tabelle base dello schema tramite *disdici*, che è poco più di un *wrapper* per l'UPDATE su *prenotazione*.

```

GRANT CONNECT, CREATE SESSION TO discente;
GRANT SELECT ON ric_disp TO discente;
GRANT EXECUTE ON prenota TO discente;
GRANT EXECUTE ON disdici TO discente;

```

L'utente *docente* ha l'onere di caricare i dati dei ricevendi e può stabilire giorno, ora e durata dei ricevimenti, oppure visualizzare l'elenco dei prenotati al prossimo ricevimento. Egli non può in alcun altro modo modificare le prenotazioni. Tutti coloro che vogliono essere ricevuti devono mandare a Monsieur De La Palisse una email preliminare di iscrizione al ricevimento, a seguito della quale egli caricherà i dati del ricevendo, di fatto abilitandone le future prenotazioni.

```

GRANT CONNECT, CREATE SESSION TO docente;
GRANT SELECT ON prenotati TO docente;
GRANT EXECUTE ON prog_ric TO docente;
GRANT ALL ON ricevendo TO docente;

```

Scheduler - facoltativo

Lo *scheduler* serve a programmare azioni, normalmente periodiche, che avvengono in istanti predeterminati. Similmente ad un trigger, vale il paradigma Evento-Condizione-Azione (ECA), ma l'evento è legato allo scorrere del tempo — quindi all'orologio di sistema — piuttosto che ad un'operazione DML. E' possibile programmare operazioni periodiche di manutenzione, *auditing*, *backup*, pulitura o aggiornamento dei dati; sia di tipo amministrativo che da utente comune.

In questo caso si crea un *job* per implementare una politica di *roll-out* dei dati: ogni primo del mese si cancellano tutte le prenotazioni e tutte le date di ricevimento fissate fino al trentesimo giorno precedente. Grazie alla politica di reazione definita su *prenotazione*, è sufficiente programmare la cancellazione delle tuple di *ricevimento*. E' mostrato come utilità anche il codice per cancellare un *job*.

```

--%%%%%%%% JOB CHE OGNI PRIMO DEL MESE CANCELLA TUTTI I RICEVIMENTI
PRECEDENTI DI UN MESE
BEGIN
DBMS_SCHEDULER.CREATE_JOB (

```



```
job_name      => 'Rollout',
job_type      => 'PLSQL_BLOCK',
job_action    => 'BEGIN
                DELETE FROM ricevimento
                WHERE data_e_ora_i<SYSDATE-30;
            END;',
start_date    => TO_DATE('01-SET-2014','DD-MON-YYYY'),
repeat_interval => 'FREQ=MONTHLY',
enabled       => TRUE,
comments      => 'Cancellazione dei dati vecchi');
END;

--%%%%%%%% CANCELLAZIONE DEL JOB ROLLOUT
BEGIN
DBMS_SCHEDULER.DROP_JOB ('Rollout');
END;
```